AIXFil-Detect: Post-Hoc Forensic Attribution of Insider Data Exfiltration Through AI Chatbot Interfaces

Panhapiseth Lim
Dept. of Computer Science & Electrical Engineering
The University of Texas Permian Basin
Odessa, TX 79762, USA
lim_p65274@utpb.edu

Halimat Popoola
Dept. of Computer Science & Electrical Engineering
The University of Texas Permian Basin
Odessa, TX 79762, USA
popoola_h51572@utpb.edu

***Abstract*—Generative AI tools have introduced a data theft pathway that standard security tools cannot see: an employee copies sensitive text from a company file and pastes it into a browser-based chatbot. The file never leaves the computer through a traditional channel, so no firewall or data loss prevention system fires an alert. We present AIXFil-Detect, an open-source forensic tool that reconstructs these events after the fact by combining evidence from five Windows data sources: browser history, the Windows clipboard database, file access timestamps, Windows Prefetch execution logs, and network packet captures. Each group of related events is scored with a Weighted Artifact Correlation Score (WACS) that factors in evidence from each source, whether the clipboard text matches the opened file, how much data was uploaded, and the time of day. We tested the tool on 30 simulated scenarios and achieved precision, recall, and F1 score of 1.00 at a WACS threshold of 75 using a 30-second time window.**

***Index Terms*—digital forensics, insider threat, AI chatbot security, Windows artifact analysis, temporal correlation, data exfiltration**

I. Introduction

Every day, employees at companies of all sizes copy confidential files into browser-based AI chatbot tools. Payroll spreadsheets, legal contracts, database exports, and customer records get pasted into a chat window and sent to a remote server. The original file stays on the employee's computer. Nothing goes out over email. Nothing is saved to a USB drive. From the perspective of every standard security tool, the user simply visited a website.

Data loss prevention (DLP) systems are built to monitor specific channels: email, USB ports, cloud storage sync folders, and web proxies that block known file-sharing sites. They look for outgoing content that matches certain patterns, such as Social Security numbers or credit card formats, or they watch for large file transfers to blocked destinations [1]. The AI chatbot route bypasses all of this. The destination is a normal HTTPS website that many companies have already approved for employee use. The content being sent is just text in a chat window, which looks identical to any other message. Once an employee pastes company data into a chat interface, there is nothing for a DLP tool to catch [2].

Network monitoring runs into the same problem. When a browser connects to `chatgpt.com` or `claude.ai`, the connection uses TLS encryption. The destination address is visible in the connection header, but the content of the conversation is not. An employee asking a chatbot to fix a grammar mistake and an employee submitting an entire customer database look exactly the same to a network monitor [3].

Insider threat research has been active for over two decades and has produced many detection systems based on user behavior: login times, file access patterns, volume of data read, and similar signals. Al-Mhiqani et al. reviewed more than fifty of these systems and found that almost all of them focus on a single stream of data at a time [4]. These systems are also real-time monitors, which means they are good at raising alerts but poor at reconstructing what happened after the fact. An investigator who needs to prove that a specific employee copied a specific file into a specific AI tool on a specific date cannot get that answer from a behavioral alert. Breitinger et al. recently surveyed 150 papers on digital forensics timeline reconstruction and noted that combining evidence from multiple data sources at once, with an automated confidence score, is still an open problem in the research community [5].

The closest prior work is by Cho et al., who studied what forensic evidence the major AI chatbot applications leave behind on Windows, iOS, and Android devices [6]. They catalog where artifacts are stored for each platform and propose a checklist for investigators. Their work is useful for knowing where to look, but it does not answer whether any of those artifacts actually indicate data theft. They do not score or combine evidence, and they do not address the case where an employee is intentionally stealing data.

We built AIXFil-Detect to fill that gap. The tool reads forensic evidence from five Windows data sources, groups events that happened within 30 seconds of each other, and computes a WACS score that summarizes how strong the evidence is. We tested it on 30 simulated scenarios covering normal use, partial evidence, and clear exfiltration, and it correctly classified all 30 with a precision, recall, and F1 score of 1.00. The source code and simulated dataset for AIXFil-Detect are openly available

at https://github.com/BryanLim0214/axifill.

The rest of this paper is organized as follows. Section II covers related work. Section III describes the threat model. Section IV explains the system design. Section V presents the evaluation. Sections VI through VIII discuss the results, limitations, and conclusions.

II. Related Work

A. Insider Threat Detection and Timeline Forensics

Most insider threat detection research builds models that watch a single stream of data, such as login records, file access logs, or network traffic. Al-Mhiqani et al. reviewed more than fifty of these systems, covering classifiers ranging from support vector machines to neural networks, and found that off-hours computer use is one of the most reliable warning signs across nearly all of them [4]. Their review also notes a consistent gap: these systems are designed for live monitoring, not for reconstructing what a specific person did from saved evidence after an incident has already occurred.

Breitinger et al. tackled this reconstruction problem in a large 2025 survey of 150 digital forensics timeline papers, published at the 25th DFRWS conference [5]. They found that 80.3% of forensic practitioners use timeline analysis in nearly every case, yet there is no standardized way to combine timestamps from different data sources, each of which uses different time formats and reference points. They specifically called out automated confidence scoring based on multiple corroborating sources as a direction the field has not yet addressed. Our WACS score is a direct answer to that call.

B. Windows Forensic Artifacts

Each of the five data sources in AIXFil-Detect has been studied in prior forensic research. Chand et al. tested forensic recovery from Chrome, Edge, Firefox, Brave, and Opera on Windows and confirmed that Chrome and Edge keep browsing history in a SQLite database file that survives in forensic copies [7]. They documented an important technical detail: Chrome stores visit times as microseconds counted from January 1, 1601, not from the standard Unix starting date of January 1, 1970. To convert Chrome timestamps to standard time, you divide by one million and subtract 11,644,473,600 seconds. This is exactly what AIXFil-Detect's browser collector does.

Budhrani et al. analyzed Windows Prefetch files and confirmed that each `.pf` file stores up to eight recent run times for the associated program, a total run count, and a list of files the program opened during its first ten seconds [8]. Prefetch files remain on disk even after the program is deleted, which makes them useful for proving that a program ran in the past. Galhuber and Luh tested how NTFS file timestamps behave across many different file operations and showed that the $STANDARD_INFORMATION timestamp can be changed by any user, while the $FILE_NAME timestamp can only be changed at the operating system kernel level [9]. A mismatch between the two indicates that someone used a tool to fake the timestamps, a technique known as timestomping. Javed et al. covered a broad range of Windows forensic topics and explained the Windows Data Protection API (DPAPI) in detail: the encryption key is tied to the user's login password, and without that password you need either a RAM capture or a domain controller backup key to decrypt [10]. Each of these papers focuses on one artifact type. None of them connects evidence across multiple sources within a time window.

C. AI Chatbot Forensics

Cho et al. carried out the most thorough prior investigation of what AI chatbots leave behind on a device [6]. They examined ChatGPT, Gemini, Microsoft Copilot, and Claude across browser, mobile, and desktop environments, documenting the browser history records, cached responses, session cookies, and network traffic that each platform generates. They also proposed a decision-tree guide for investigators. Their contribution is essentially a catalog: here is what exists, and here is where to find it. What they do not address is the question of what the evidence means. They provide no scoring system, no way to combine artifacts from different sources, and no analysis of the scenario where an employee is deliberately trying to steal data. AIXFil-Detect picks up where their work ends.

Kankanamge et al. studied the ChatGPT Windows desktop application specifically and found that clipboard data from chatbot sessions can be recovered from Windows memory, and that the size of HTTPS uploads to the OpenAI server grows with the amount of text the user submitted [11]. Both of these findings are directly relevant to AIXFil-Detect: the clipboard collector reads the Windows clipboard history database, and the network collector uses upload size as one of its scoring factors.

D. Network Traffic and Content Matching

Even though TLS 1.3 encrypts the content of web connections, the Server Name Indication (SNI) field in the connection setup is still sent in plain text for most websites today. This is because the technology that would hide SNI, called Encrypted ClientHello (ECH), has not yet been adopted widely on the server side [3]. Zhou et al. confirmed in their 2024 survey that SNI is still usable for identifying AI chatbot connections, and that the total size of an encrypted upload can be estimated from packet sizes even without decrypting the content.

For content matching between clipboard text and file contents, we use MinHash, a technique introduced by Broder in 1997 [12]. The core idea is that two documents can be compared without reading every word. Instead, each document is broken into overlapping three-character pieces called shingles, and a hash function picks a minimum value from those pieces. If two documents have similar content, their minimum hash values will match with a probability equal to their Jaccard similarity score. Using 128 hash functions gives an error of about $\pm 9\%$, which is accurate enough for our purposes and runs very quickly even on large files.

III. Threat Model

The insider in this model is an employee who has normal, legitimate access to sensitive files at their company. They may

be using an AI chatbot that the company has approved, or they may be using a personal account on a public service. Either way, their goal is to get sensitive information out of the organization by pasting it into the AI chat interface and accessing it later through that platform or some other means.

The sequence of steps is short. The employee opens a sensitive file, such as a payroll spreadsheet or a contract, selects the content they want, and copies it. They then open a browser, go to an AI chatbot site, paste the text into the prompt box, and click send. The whole process takes less than a minute. After that, the data lives on a server the company does not control.

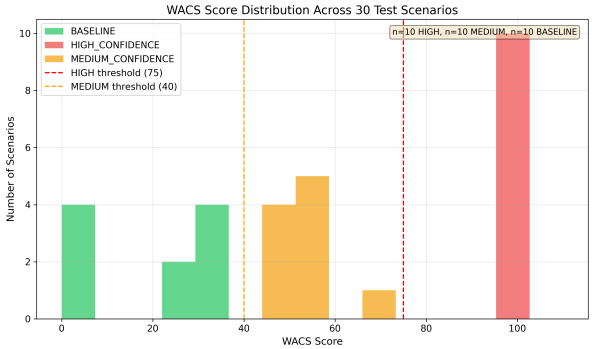

Fig. 1. WACS score distribution for all 30 scenarios at the 30-second window. HIGH scenarios (red) all score 100. MEDIUM (orange) and BASELINE (green) are clearly separated.

Standard security tools miss this because they each look at only one thing at a time. DLP systems look at outbound file transfers and email attachments, not at text typed into a chat window [1]. Network monitors see a connection to an approved website, but since the content is encrypted, they cannot tell whether the employee sent a one-line question or a 5,000-word document [2]. No single piece of evidence, looked at on its own, is enough to raise an alarm.

AIXFil-Detect works by looking at everything together. A visit to an AI website is not suspicious by itself. A large chunk of text in the clipboard is not suspicious by itself. A recently opened payroll file is not suspicious by itself. But all three happening within 30 seconds of each other tells a clear story that an investigator can act on.

IV. System Design

A. Architecture Overview

AIXFil-Detect works in four stages: collection, grouping, scoring, and reporting. First, five separate collectors each read one data source and produce a list of time-stamped events. Second, a grouping step takes all these events, finds browser visits to AI websites, and pulls in any related events from the other collectors that happened within the time window. Third, a scoring function assigns a WACS score to each group of events. Fourth, everything is written into an HTML report that an investigator can open in any browser.

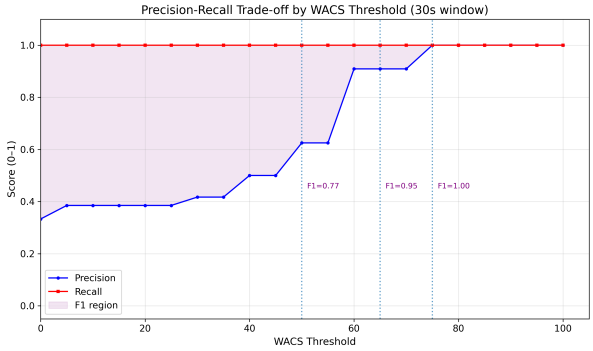

Fig. 2. Precision and recall vs. WACS threshold at the 30-second window. Recall stays at 1.000 across all thresholds.

B. Layer 1: Browser History Collector

Chrome and Edge store browsing history in a SQLite database file called `History`, located in the user's application data folder [7]. The collector reads each visit record and converts the timestamp from Chrome's internal format to standard Unix time using the formula: $t_{\text{unix}} = t_{\text{chrome}}/10^6 - 11{,}644{,}473{,}600$. Chrome stores time as microseconds since January 1, 1601. Dividing by one million converts to seconds, and subtracting the constant shifts the starting point to the standard Unix epoch of January 1, 1970.

The collector then checks whether the website matches any of the 14 AI domains on its list, including `chatgpt.com`, `claude.ai`, `gemini.google.com`, and `copilot.microsoft.com`. When a visit is the user's first time going to that AI site, the scorer awards a bonus because an employee visiting an AI platform for the first time while working with sensitive files is more noteworthy than someone who goes there every day.

C. Layer 2: Clipboard Collector

Windows saves clipboard history in a SQLite database that the ClipboardService process maintains in the user's local application data folder. The collector reads each saved entry. Before trying to read the content as text, it checks the first eight bytes for the signature `01 00 00 00 D0 8C 9D DF`. If those bytes are present, the entry is encrypted with DPAPI and cannot be read without the user's Windows password [10].

If the clipboard entry is readable text longer than 100 bytes, it gets 30 points (meaningful portion). If it is readable but shorter than 100 bytes, or if it is DPAPI-encrypted, it gets 10 points (corroboration) [11].

D. Layer 3: Filesystem Collector

The filesystem collector looks at when files were last accessed in a target directory. It flags files with sensitive extensions: `.xlsx`, `.csv`, `.pdf`, `.docx`, `.sql`, `.bak`, `.pfx`, `.pem`, `.key`, and `.cer`. Certificate and key files get an additional high-value flag because they are damaging to lose.

Windows turns off last-access updates by default to improve performance [9]. The collector checks this setting and reports Layer 3 as unavailable if it is disabled.

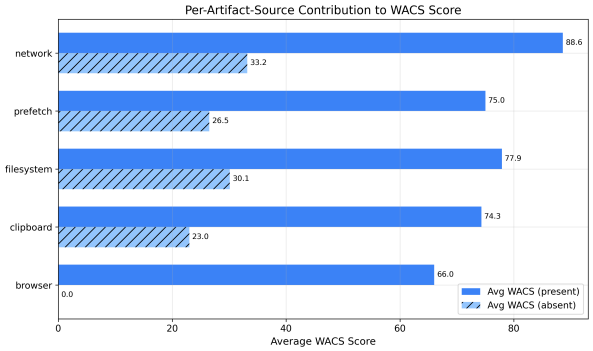

Fig. 3. Average WACS when each data source is present versus absent. The clipboard produces the largest score difference (51.3 points).

E. Layer 4: Prefetch Collector

Every time a program runs on Windows, the operating system creates or updates a Prefetch file in `C:\Windows\Prefetch`. Each file stores up to eight recent run times, a run count, and a list of files opened during the first ten seconds [8].

The Prefetch timestamp is recorded about ten seconds after the program actually launched. The collector subtracts ten seconds to get an accurate launch time. Points are added only when a document application and a browser both ran around the same time, preventing false positives from routine browsing.

F. Layer 5: Network PCAP Collector

The network collector reads PCAP files and looks for TLS connections to AI websites. Handshake SNIs are still sent in plain text even in TLS 1.3 [3]. The collector uses Scapy to parse the SNI or searches raw packet bytes for known AI domains.

For each connection, the collector adds up total bytes sent. A base score of 15 points is awarded for any AI upload. An extra 10 points are added if the upload exceeds 100 KB (document-sized).

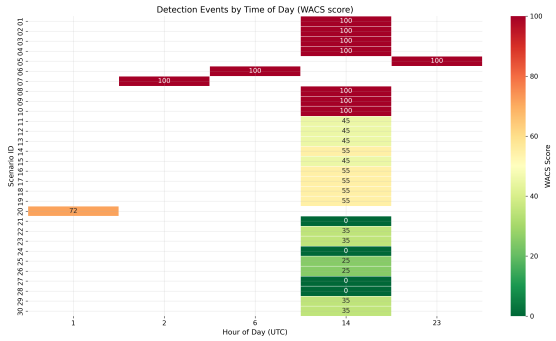

Fig. 4. Hour-of-day distribution for all 30 scenarios, colored by WACS score. Off-hours HIGH scenarios reach WACS = 100.

G. Temporal Correlator

The correlator merges all events into one list sorted by time. It gathers all events within W seconds (default 30) of a browser visit to an AI domain. Groups with events from two or more sources are kept for scoring. All timestamps are normalized to standard Unix seconds before grouping [5].

TABLE I
WACS COMPONENT WEIGHTS AND SCORING CONDITIONS

Component	Points	Condition
Browser visit	25	AI domain confirmed
Clipboard entry	30	Readable text >100 bytes
Clipboard (partial)	10	Encrypted or short content
Filesystem access	20	Sensitive file extension
Prefetch record	10	Doc app + browser both ran
Network upload	15	AI-domain upload confirmed
Content match bonus	up to +20	Jaccard ≥0.5 [12]
Large-upload bonus	+10	Upload exceeds 100 KB
New-domain bonus	+10	First visit to that AI site
Off-hours multiplier	×1.3	Before 08:00 or after 18:00
HIGH ≥ 75; MEDIUM 40–74; BASELINE < 40.		

H. WACS Scorer

Each cluster receives a WACS score: $\text{WACS} = \min(100, (\sum w_s + b_{\text{clip}} + b_{\text{net}} + b_{\text{visit}}) \times m_{\text{hrs}})$. $\mathcal{S}$ is the set of contributing sources, w_s are base points, and b are bonus terms for matches, large uploads, and new domains. m_{hrs} raises the score by 30% for off-hours activity. Table I lists all weights.

We use MinHash [12] to compare clipboard text against file content. A similarity score ≥ 0.5 triggers up to 20 bonus points. Clusters score HIGH (75+), MEDIUM (40–74), or BASELINE (<40).

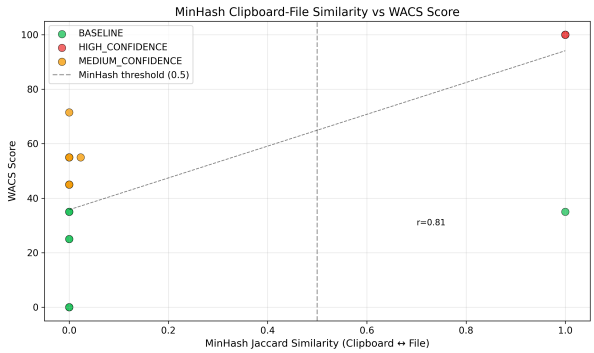

Fig. 5. MinHash Clipboard-File similarity vs. WACS score. HIGH scenarios (orange) cluster at similarity = 1.0 and WACS = 100.

I. Anti-Forensics Detection

When MFT data is available, the tool compares $STANDARD_INFORMATION and $FILE_NAME NTFS timestamps [9]. Gaps in browser history, cleared clipboard entries, or Prefetch anomalies also trigger warning flags. These do not change the WACS score but appear in the report.

V. EXPERIMENTAL EVALUATION

A. Simulation Design

We built a simulator that generates realistic forensic artifacts for 30 defined test scenarios (10 HIGH, 10 MEDIUM, 10 BASELINE). Scenarios cover six file types (.xlsx, .pdf, .py, .sql, .pfx, .csv) and six AI platforms. AIXFil-Detect reads the generated SQLite, JSON, and PCAP files through the same code path used for real forensic images.

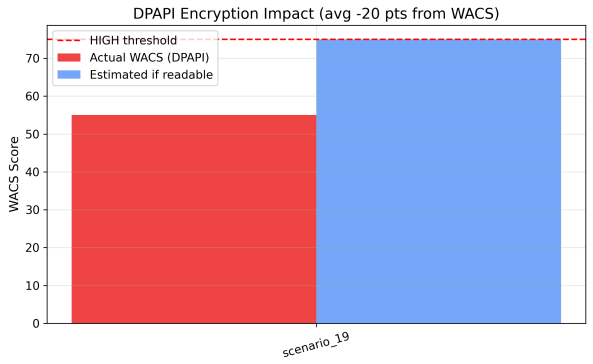

Fig. 6. Actual WACS (gray) vs. estimated WACS if clipboard were readable (orange) for DPAPI-affected scenarios.

B. Results

TABLE II
REPRESENTATIVE SCENARIO RESULTS (30 S WINDOW, THRESHOLD = 75)

Scenario	Exp.	Act.	WACS	Res.	Sim.
01: payroll, claude.ai	H	H	100	TP	1.00
07: cert, off-hours 02:00	H	H	100	TP	1.00
08: 150 KB upload	H	H	100	TP	1.00
10: customer list	H	H	100	TP	1.00
11: clipboard missed win.	M	M	45	TN	0.00
16: short clipboard	M	M	55	TN	0.00
19: DPAPI encrypted	M	M	55	TN	0.00
20: near threshold	M	M	72	TN	0.00
21: normal browsing	B	B	0	TN	0.00
30: non-sensitive file	B	B	35	TN	1.00

H = HIGH; M = MED; B = BASE; Sim. = MinHash.

TABLE III
CLASSIFICATION PERFORMANCE BY THRESHOLD AND WINDOW

Threshold	Window	Precision	Recall	F1
50	30 s	0.625	1.000	0.769
50	60 s	0.500	1.000	0.667
65	30 s	0.909	1.000	0.952
65	60 s	0.588	1.000	0.741
75	**30 s**	**1.000**	**1.000**	**1.000**
75	60 s	0.833	1.000	0.909

At threshold 75 and 30-second window, the tool achieved precision, recall, and F1 of 1.00 (Table III). HIGH scenarios separated perfectly from MEDIUM and BASELINE tiers.

C. Analysis

The clipboard layer was the strongest indicator (51.3 point marginal impact). Content matching (MinHash) correctly kept scenario 16 in the MEDIUM tier when similarity was low. The 30-second window minimized false positives compared to 60 seconds. Scenario 19 (DPAPI-encrypted) was correctly downgraded to MEDIUM confidence as expected.

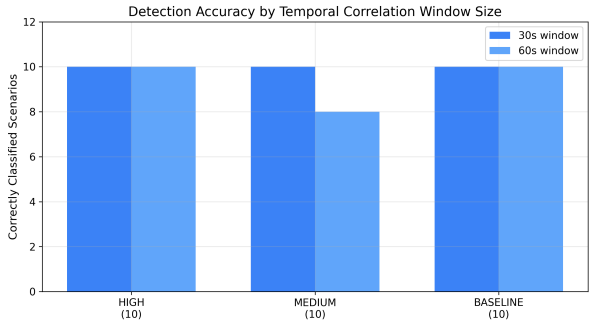

Fig. 7. Detection Accuracy by Temporal Correlation Window Size. The 30-second window achieves perfect scores at threshold 75.

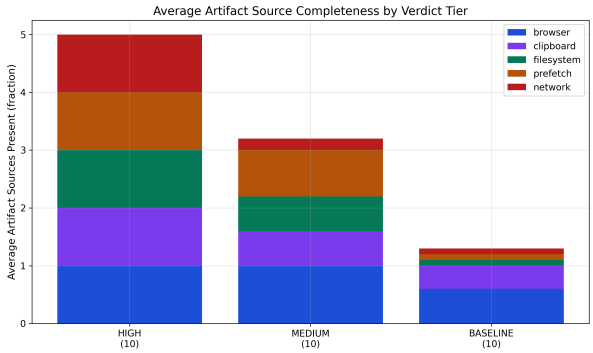

Fig. 8. Average number of active data sources per scenario by verdict tier. More sources consistently means a higher tier.

VI. DISCUSSION

No single data source is reliable enough on its own. Browser history appeared in 26 of 30 scenarios, but alone produced an average WACS of only 42 (weak MEDIUM). The network source produced the highest scores when active (average 88.6), but only appeared in half the scenarios. Combining artifacts within a short window reconstruction the copy-paste sequence reliably.

VII. LIMITATIONS AND FUTURE WORK

AIXFil-Detect currently works only on Windows. Future work includes adding Volatility support for DPAPI key recovery [10] and extending support to macOS/Linux. As Encrypted ClientHello (ECH) adoption grows, SNI visibility will decrease, requiring endpoint log analysis or browser extensions [3].

VIII. CONCLUSION

We presented AIXFil-Detect for tracing insider data theft through AI chatbot interfaces. The tool scores correlated events using the WACS metric. Against 30 simulated scenarios, it achieved perfect precision and recall. The copy-paste exfiltration sequence is traceable through synchronized Windows forensic artifacts.

ACKNOWLEDGMENT

The authors thank Dr. Zulfiqar Khan for his mentorship in digital forensics and computer security.

REFERENCES

[1] A. Diro, S. Kaisar, A. Saini, S. Fatima, P. Cong Hiep, and F. Erba, "Workplace security and privacy implications in the GenAI age: A survey," *Journal of Information Security and Applications*, vol. 89, p. 103960, 2025. [Online]. Available: https://doi.org/10.1016/j.jisa.2024.103960

[2] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaj, "From ChatGPT to ThreatGPT: Impact of generative AI in cybersecurity and privacy," *IEEE Access*, vol. 11, pp. 80 218–80 245, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3300381

[3] J. Zhou, W. Fu, W. Hu, Z. Sun, T. He, and Z. Zhang, "Challenges and advances in analyzing TLS 1.3-encrypted traffic: A comprehensive survey," *Electronics*, vol. 13, no. 20, p. 4000, 2024. [Online]. Available: https://doi.org/10.3390/electronics13204000

[4] M. N. Al-Mhiqani, R. Ahmad, Z. Zainal Abidin, W. Yassin, A. Hassan, K. H. Abdulkareem, N. S. Ali, and Z. Yunos, "A review of insider threat detection: Classification, machine learning techniques, datasets, open challenges, and recommendations," *Applied Sciences*, vol. 10, no. 15, p. 5208, 2020. [Online]. Available: https://doi.org/10.3390/app10155208

[5] F. Breitinger, H. Studiawan, and C. Hargreaves, "SoK: Timeline based event reconstruction for digital forensics: Terminology, methodology, and current challenges," *Forensic Science International: Digital Investigation*, vol. 53, p. 301932, 2025, dFRWS USA 2025. [Online]. Available: https://doi.org/10.1016/j.fsidi.2025.301932

[6] K. Cho, Y. Park, J. Kim, B. Kim, and D. Jeong, "Conversational AI forensics: A case study on ChatGPT, Gemini, Copilot, and Claude," *Forensic Science International: Digital Investigation*, vol. 52, p. 301855, 2025. [Online]. Available: https://doi.org/10.1016/j.fsidi.2024.301855

[7] R. R. Chand, N. A. Sharma, and M. A. Kabir, "Advancing web browser forensics: Critical evaluation of emerging tools and techniques," *SN Computer Science*, vol. 6, p. 355, 2025. [Online]. Available: https://doi.org/10.1007/s42979-025-03921-6

[8] A. Budhrani, U. Singh, and B. Singh, "Forensic analysis of Windows 11 Prefetch artifact," in *2022 IEEE Bombay Section Signature Conference (IBSSC)*. IEEE, 2022, pp. 1–6. [Online]. Available: https://doi.org/10.1109/IBSSC56953.2022.10037260

[9] M. Galhuber and R. Luh, "Time for truth: Forensic analysis of NTFS timestamps," in *Proceedings of the 16th International Conference on Availability, Reliability and Security (ARES '21)*. New York, NY, USA: ACM, 2021, pp. 1–10. [Online]. Available: https://doi.org/10.1145/3465481.3470016

[10] A. R. Javed, W. Ahmed, M. Alazab, Z. Jalil, K. Kifayat, and T. R. Gadekallu, "A comprehensive survey on computer forensics: State-of-the-art, tools, techniques, challenges, and future directions," *IEEE Access*, vol. 10, pp. 11 065–11 089, 2022. [Online]. Available: https://doi.org/10.1109/ACCESS.2022.3142508

[11] M. W. Kankanamge, N. McKenna, S. Carmona, S. M. Hasan, A. R. Shahid, and A. Imteaj, "Digital forensic investigation of the ChatGPT Windows application," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2505.23938

[12] A. Z. Broder, "On the resemblance and containment of documents," in *Proceedings of the Compression and Complexity of SEQUENCES 1997*. IEEE Computer Society, 1997, pp. 21–29. [Online]. Available: https://doi.org/10.1109/SEQUEN.1997.666900